Title: Web Server Compromise & Detection Project

Name: R Ashwin

Role: Cybersecurity Enthusiast | SOC Analyst Aspirant

Date: 18.04.2026

Web Server Compromise & Detection

Objective

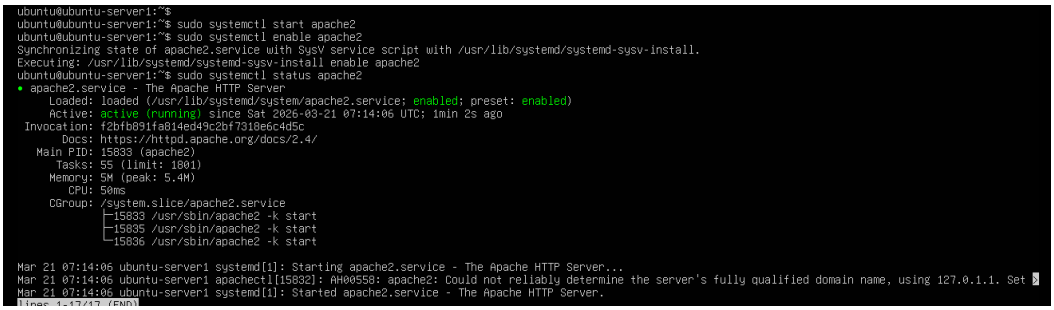
To simulate real-world web application attacks and detect them using centralized log monitoring, demonstrating how SOC analysts identify and analyze web-based threats.

Tools Used

- Apache Web Server
- DVWA (Damn Vulnerable Web Application)
- MySQL / MariaDB
- rsyslog
- Kali Linux
- Nmap

Web Server Setup

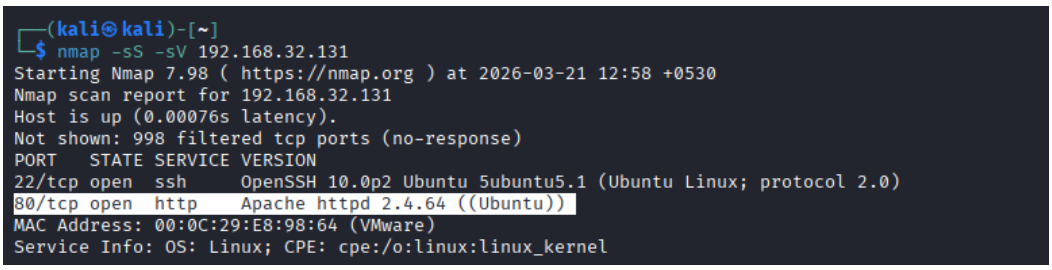
Apache web server was installed and configured on the victim machine. Port 80 was verified to be open, and the web application was accessible through a browser.



```
ubuntu@ubuntu-server1:~$
ubuntu@ubuntu-server1:~$ sudo systemctl start apache2
ubuntu@ubuntu-server1:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
ubuntu@ubuntu-server1:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-03-21 07:14:06 UTC; 1min 2s ago
 Invocation: f2bfb891fa814ed49c2bf7318e6c4d5c
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 15833 (apache2)
      Tasks: 55 (limit: 1801)
     Memory: 5M (peak: 5.4M)
        CPU: 50ms
     CGroup: /system.slice/apache2.service
             ├─15833 /usr/sbin/apache2 -k start
             ├─15835 /usr/sbin/apache2 -k start
             └─15836 /usr/sbin/apache2 -k start

Mar 21 07:14:06 ubuntu-server1 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 21 07:14:06 ubuntu-server1 apachectl[15832]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set
Mar 21 07:14:06 ubuntu-server1 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-17/17 (END)
```

Figure 1: Apache service installed and confirmed active.



```
┌──(kali㉿kali)-[~]
└─$ nmap -sS -sV 192.168.32.131
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-21 12:58 +0530
Nmap scan report for 192.168.32.131
Host is up (0.00076s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.0p2 Ubuntu 5ubuntu5.1 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.64 ((Ubuntu))
MAC Address: 00:0C:29:E8:98:64 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 2: Nmap scan showing port 80 open.

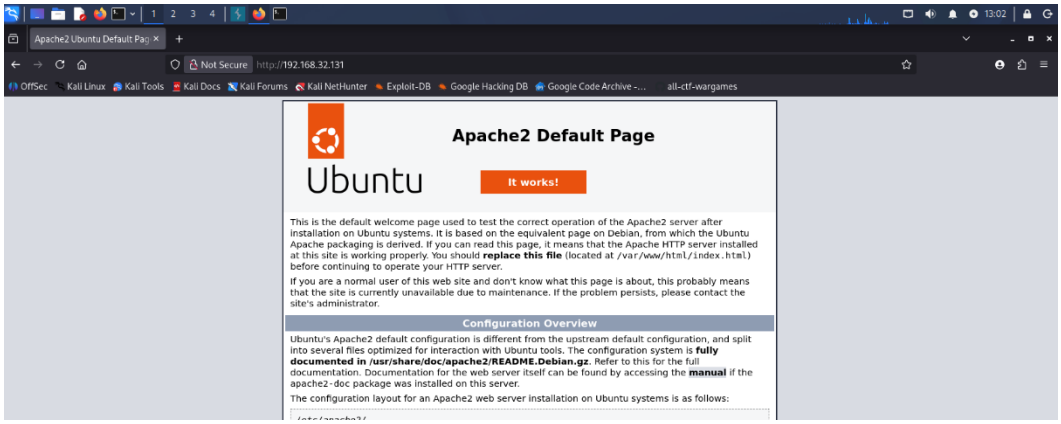


Figure 3: Web browser successfully loading the application page.

DVWA Setup

DVWA (Damn Vulnerable Web Application) was installed to simulate real-world web vulnerabilities such as SQL injection, command injection, and file upload attacks.

```
ubuntu@ubuntu-server1:~$
ubuntu@ubuntu-server1:~$ cd /var/www/html
ubuntu@ubuntu-server1:/var/www/html$ sudo git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5703, done.
remote: Counting objects: 100% (52/52), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 5703 (delta 27), reused 24 (delta 16), pack-reused 5651 (from 2)
Receiving objects: 100% (5703/5703), 2.72 MiB | 6.64 MiB/s, done.
Resolving deltas: 100% (2842/2842), done.
ubuntu@ubuntu-server1:/var/www/html$ _
```

Figure 4: DVWA installed in /var/www/html .

```
ubuntu@ubuntu-server1:~$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 11.8.3-MariaDB-1build1 from Ubuntu -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> ALTER USER 'root'@'localhost' IDENTIFIED BY 'rootpass';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
ubuntu@ubuntu-server1:~$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 33
Server version: 11.8.3-MariaDB-1build1 from Ubuntu -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]>
```

Figure 5: MariaDB configuration.

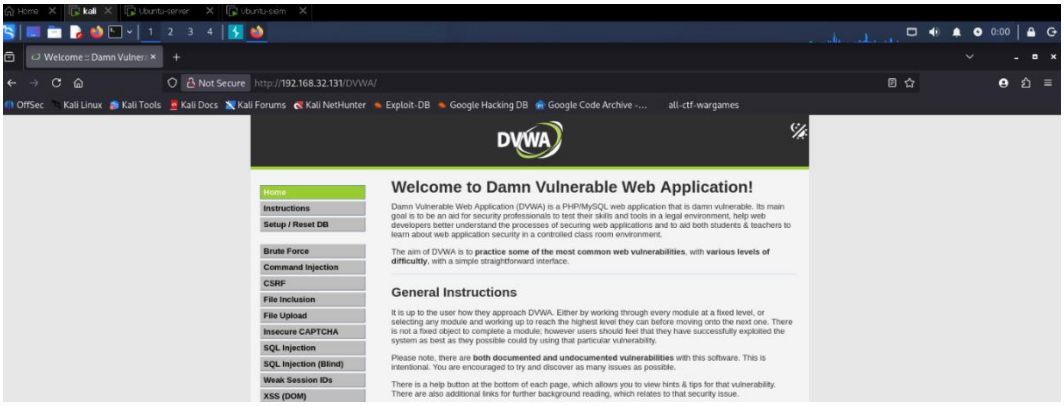


Figure 6: Confirmed the working of DVWA Dashboard.

Reconnaissance

- The attacker performed reconnaissance using tools like Nmap and directory enumeration techniques to identify open services and hidden endpoints.
- This phase helps attackers map the attack surface before exploitation.

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 80 -sV 192.168.32.131
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-22 00:05 +0530
Nmap scan report for 192.168.32.131
Host is up (0.00037s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.64 ((Ubuntu))
MAC Address: 00:0C:29:E8:98:64 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.04 seconds
```

Figure 7: Website service detection using Nmap.

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.32.131/DVWA -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.32.131/DVWA
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 279]
.git/HEAD            (Status: 200) [Size: 23]
.htpasswd            (Status: 403) [Size: 279]
.htaccess            (Status: 403) [Size: 279]
config               (Status: 301) [Size: 322] [--> http://192.168.32.131/DVWA/config/]
database             (Status: 301) [Size: 324] [--> http://192.168.32.131/DVWA/database/]
docs                 (Status: 301) [Size: 320] [--> http://192.168.32.131/DVWA/docs/]
external             (Status: 301) [Size: 324] [--> http://192.168.32.131/DVWA/external/]
favicon.ico          (Status: 200) [Size: 1406]
index.php            (Status: 302) [Size: 0] [--> login.php]
php.ini              (Status: 200) [Size: 154]
phpinfo.php          (Status: 302) [Size: 0] [--> login.php]
robots.txt           (Status: 200) [Size: 25]
tests                (Status: 301) [Size: 321] [--> http://192.168.32.131/DVWA/tests/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 8: Discovered directories using gobuster.

Attack Sections

1. Command Injection

- Attack
  - The attacker exploited a command injection vulnerability by appending system commands to user input fields.
- Payloads
  - 127.0.0.1; whoami
  - 127.0.0.1; id
  - 127.0.0.1; uname -a
  - 127.0.0.1; ls
- Execution & Results

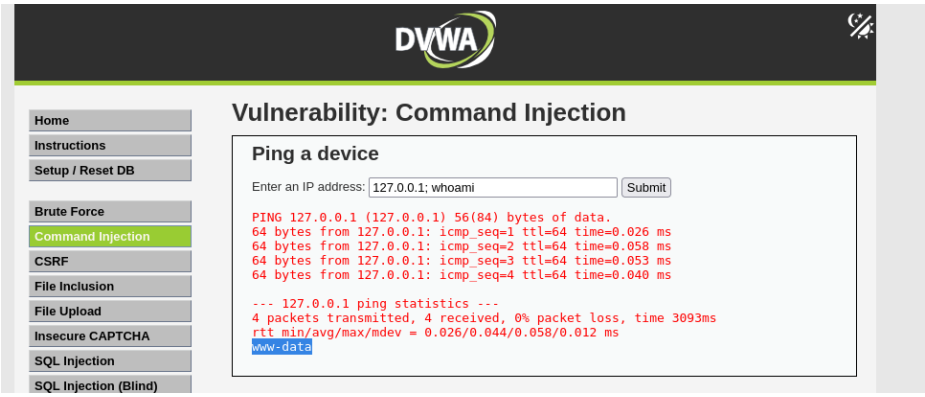


Figure 9: Command injection executing system-level command (*whoami)* demonstrating user context access.

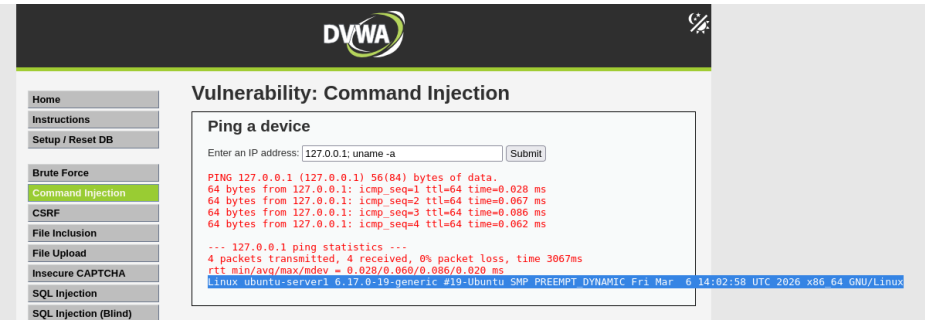


Figure 10: *Command injection retrieving system information (uname -a) confirming full command execution capability.*

- Result
  - The server executed arbitrary system commands and returned output.
- Detection
  - Apache logs recorded incoming requests; however, the malicious payload was not clearly visible, highlighting limitations in default logging.

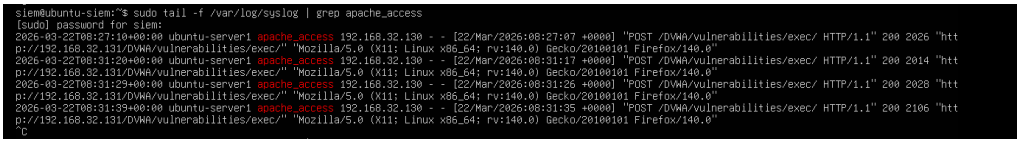


Figure 11: SIEM logs from rsyslog showing the signs of possible web attack.

2. SQL Injection

- Attack
  - SQL queries were injected into input fields to manipulate backend database queries.
- Payloads
  - 1' OR '1'='1
  - UNION SELECT null, version()#
  - 1' UNION SELECT user, password FROM users#
- Execution & Results

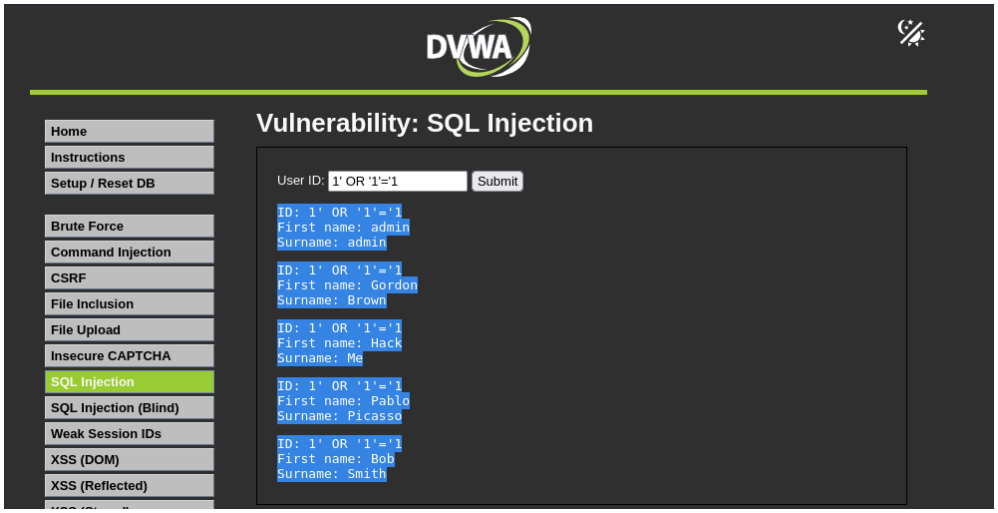


Figure 12: SQL Injection payload (1' OR '1'='1) retrieves all records from the database.

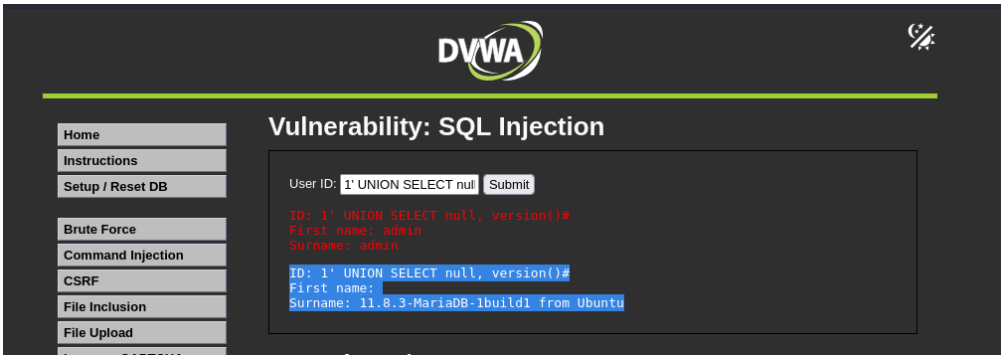


Figure 13: SQL Injection payload (UNION SELECT null, version()#)retrieves version of the DBMS.

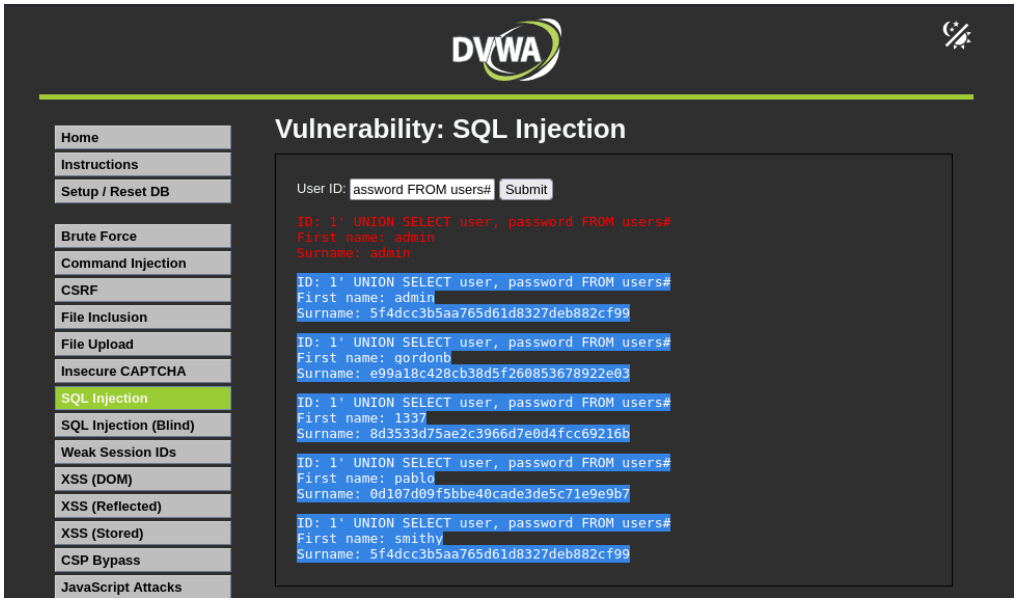
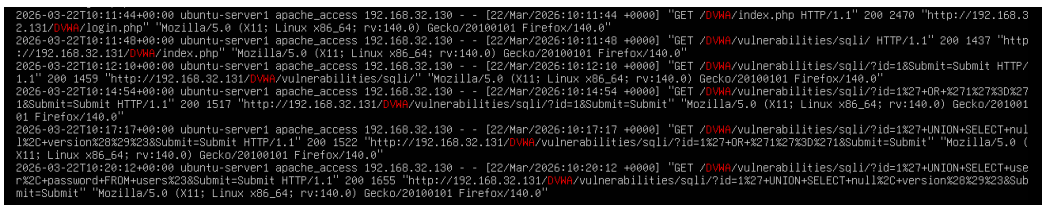


Figure 14: SQL Injection payload (1' UNION SELECT user, password FROM users#) retrieves every user and their respective password.

- Result
  - Unauthorized database access was achieved, returning sensitive user data.
- Detection
  - Logs clearly showed SQL keywords and patterns in query strings, enabling detection of injection attempts.



```
2026-03-22T10:11:44+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:11:44 +0000] "GET /DVWA/index.php HTTP/1.1" 200 2470 "http://192.168.3
2.131/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-03-22T10:11:48+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:11:48 +0000] "GET /DVWA/vulnerabilities/sqli/ HTTP/1.1" 200 1437 "http
://192.168.32.131/DVWA/index.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-03-22T10:12:10+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:12:10 +0000] "GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/
1.1" 200 1459 "http://192.168.32.131/DVWA/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-03-22T10:14:54+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:14:54 +0000] "GET /DVWA/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%27
1&Submit=Submit HTTP/1.1" 200 1517 "http://192.168.32.131/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/201001
01 Firefox/140.0"
2026-03-22T10:17:17+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:17:17 +0000] "GET /DVWA/vulnerabilities/sqli/?id=1%27+UNION+SELECT+nul
l%2C+version%28%29%23&Submit=Submit HTTP/1.1" 200 1522 "http://192.168.32.131/DVWA/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit" "Mozilla/5.0 (
X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-03-22T10:20:12+00:00 ubuntu-server1 apache_access 192.168.32.130 - - [22/Mar/2026:10:20:12 +0000] "GET /DVWA/vulnerabilities/sqli/?id=1%27+UNION+SELECT+use
r%2C+password+FROM+users%23&Submit=Submit HTTP/1.1" 200 1655 "http://192.168.32.131/DVWA/vulnerabilities/sqli/?id=1%27+UNION+SELECT+null%2C+version%28%29%23&Sub
mit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Figure 15: Logs detecting SQLi in SIEM.

3. File Upload (Web Shell)

- Attack
  - The attacker uploaded a malicious PHP web shell to gain remote command execution.
- Payloads
  - shell.php?cmd=whoami
  - shell.php?cmd=id
  - shell.php?cmd=uname -a
  - shell.php?cmd=ls
- Execution & Results

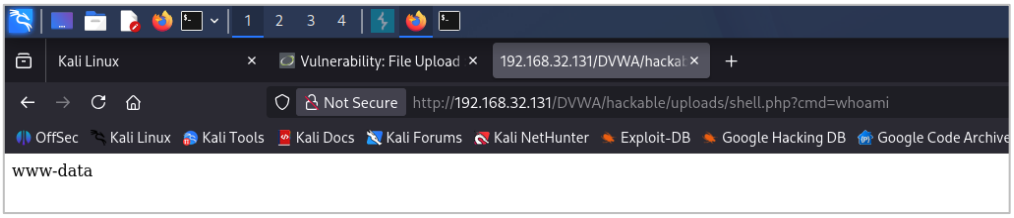


Figure 16: Web shell executing system commands (whoami) via URL parameters.

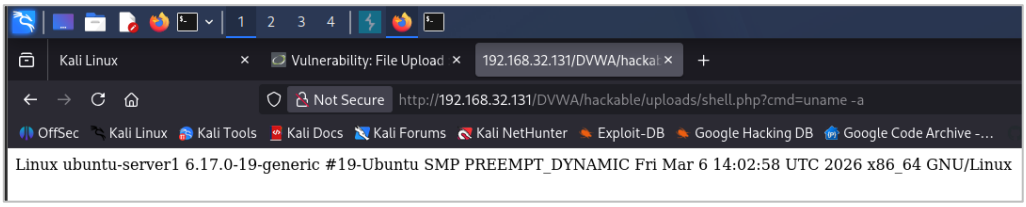


Figure 17: Remote command execution (uname -a) confirming full system access.

- Result
  - The attacker executed system commands remotely through the web interface.
- Detection
  - Logs revealed suspicious requests containing command execution parameters, indicating active exploitation.

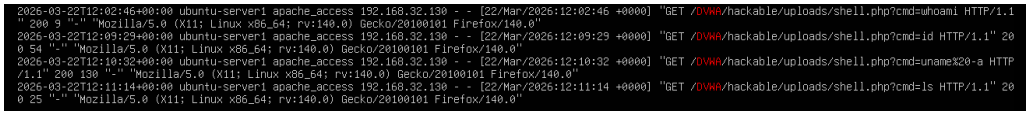


Figure 18: Logs detecting Direct command Execution via URL in SIEM server.

Detection & Analysis

The SIEM system successfully captured HTTP request logs from the web server, allowing identification of attack patterns such as:

- Repeated directory access (reconnaissance)
- Suspicious query strings (SQL injection)
- Command execution attempts via URL parameters

This demonstrates the importance of centralized logging in detecting web-based attacks. This highlights how web-based attacks can be identified through log correlation and pattern analysis.

Results

- Multiple web attacks successfully simulated
- Attack patterns detected through SIEM logs
- Limitations of default logging identified
- Web shell exploitation detected via HTTP logs

Key Learnings

- Web applications are highly vulnerable to input-based attacks
- Centralized logging is essential for detecting web threats
- Default logging may not capture full attack payloads
- Attackers use multiple techniques in sequence (recon → exploit → persistence)

Security Insights

- Web applications are highly vulnerable to input-based attacks such as SQL injection and command injection
- Centralized logging enables detection of suspicious HTTP requests and attack patterns
- Default web server logs may not capture full attack payloads, limiting visibility
- Attackers often chain multiple techniques (reconnaissance → exploitation → persistence)
- Web shells allow persistent remote command execution and are difficult to detect without proper monitoring

Conclusion

This project demonstrates how web application vulnerabilities can be exploited and how centralized log monitoring enables detection of such attacks. It highlights both the strengths and limitations of log-based detection in real-world scenarios.

Appendix: Commands Used

1. Web Server Installation (Victim)
    - `sudo apt update`
    - `sudo apt install apache2 -y`
    - Start and enable Apache:
        - `sudo systemctl enable apache2`
        - `sudo systemctl start apache2`

2. Database Setup (MariaDB)
    - `sudo apt install mariadb-server -y`
    - `sudo mariadb-secure-installation`

3. DVWA Setup (Vulnerable Web App)
    - `git clone https://github.com/digininja/DVWA.git`
    - `sudo mv DVWA /var/www/html/`
    - Set permissions:
        - `sudo chown -R www-data:www-data /var/www/html/DVWA`
        - `sudo chmod -R 755 /var/www/html/DVWA`
    - Restart Apache:
        - `sudo systemctl restart apache2`

4. Reconnaissance (Kali Linux)
    - Scan open ports:
        - `nmap -p 80 -sV 192.168.32.131`
    - Directory enumeration
        - `gobuster dir -u http://192.168.32.131/DVWA -w /usr/share/wordlists/dirb/common.txt`

5. Command Injection Payloads
    - `127.0.0.1; whoami`
    - `127.0.0.1; id`
    - `127.0.0.1; uname -a`
    - `127.0.0.1; ls`

6. SQL Injection Payloads
    - `1' OR '1'='1`
    - `UNION SELECT null, version()#`
    - `1' UNION SELECT user, password FROM users#`

7. Web Shell Execution (File Upload Attack)
    - `shell.php?cmd=whoami`
    - `shell.php?cmd=id`
    - `shell.php?cmd=uname -a`
    - `shell.php?cmd=ls`

8. Log Monitoring (SIEM Server)
    - View incoming logs:
        - `sudo tail -f /var/log/syslog`

9. Filter HTTP Logs
    - `grep "GET" /var/log/apache2/access.log`
    - `sudo tail -f /var/log/syslog | grep apache_access`

10. Service Status Check
    - `sudo systemctl status apache2`
    - `sudo systemctl status rsyslog`